16. Concurrency Control

As explained in the JDBC Transactions chapter, every SQL statement executes within the scope of a database transaction. To prevent conflicts, database engines employ row-level locks. Database physical locks can either be acquired implicitly or explicitly. Whenever a row is changed, the relational database acquires an implicit exclusive lock on the aforementioned record to prevent write-write conflicts.

Locks can also be acquired explicitly, in which case the concurrency control mechanism is called pessimistic locking. Exclusive locks can be acquired explicitly on most database systems, whereas shared locks are not universally supported.

Pessimistic locking deals with concurrency conflicts through prevention, which can impact application performance and scalability. For this reason, to increase transaction throughput while still ensuring strong consistency, many data access frameworks provide optimistic locking support as well.

16.1 Hibernate optimistic locking

Even if pessimistic locking has only been added in JPA 2.0, optimistic locking has been supported since version 1.0. Just like pessimistic locking, the optimistic concurrency control mechanism can be used implicitly or explicitly.

16.1.1 The implicit optimistic locking mechanism

To enable the implicit optimistic locking mechanism, the entity must provide a `@Version` attribute:

```
@Entity @Table(name = "post")
public class Post {

    @Id
    private Long id;

    private String title;

    @Version
    private int version;

    //Getters and setters omitted for brevity
}
```

Logical vs. Physical clocks

Using timestamps to order events is rarely a good idea. System time is not always monotonically incremented, and it can even go backward due to network time synchronization (NTP protocol).

More, time accuracy across different database systems varies from nanoseconds (Oracle[a]) to 100 nanoseconds (SQL Server[b]), to microseconds (PostgreSQL[c] and MySQL 5.6.4[d] and even seconds (previous versions of MySQL). In distributed systems, logical clocks (e.g. vector clocks or Lamport timestamps) are always preferred to physical timestamps (wall clocks) when it comes to ordering events.

For this reason, employing a numerical version is more appropriate than using a timestamp.

[a] http://docs.oracle.com/database/121/LNPCB/pco04dat.htm#LNPCB269
[b] https://msdn.microsoft.com/en-us/library/bb677335.aspx
[c] http://www.postgresql.org/docs/current/static/datatype-datetime.html
[d] http://dev.mysql.com/doc/refman/5.6/en/fractional-seconds.html

To visualize the optimistic concurrency control, when executing the following test case:

```
doInJPA(entityManager -> {
    Post post = new Post();
    post.setId(1L);
    post.setTitle("High-Performance Java Persistence");
    entityManager.persist(post);

    entityManager.flush();
    post.setTitle("High-Performance Hibernate");
});
```

Hibernate generates the following output:

```
INSERT INTO post (title, version, id)
VALUES ('High-Performance Java Persistence', 0, 1)

UPDATE post SET title = 'High-Performance Hibernate', version = 1
WHERE id = 1 AND version = 0
```

Whenever an update occurs, Hibernate is going to filter the database record according to the expected entity version. If the version has changed, the update count is going to be 0, and a `OptimisticLockException` is going to be thrown.

To visualize the conflict detection mechanism, consider the following exercise:

```
doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
    executeSync(() -> {
        doInJPA(_entityManager -> {
            Post _post = _entityManager.find(Post.class, 1L);
            _post.setTitle("High-Performance JDBC");
        });
    });
    post.setTitle("High-Performance Hibernate");
});
```

When executing the aforementioned test case, Hibernate generates the following output:

```
-- Alice selects the Post entity
SELECT p.id AS id1_0_0_, p.title AS title2_0_0_, p.version AS version3_0_0_
FROM   post p
WHERE  p.id = 1

-- Bob also selects the same Post entity
SELECT p.id AS id1_0_0_, p.title AS title2_0_0_, p.version AS version3_0_0_
FROM   post p
WHERE  p.id = 1

-- Bob updates the Post entity
UPDATE post SET title = 'High-Performance JDBC', version = 1
WHERE id = 1 AND version = 0

-- Alice also wants to update the Post entity
UPDATE post SET title = 'High-Performance Hibernate', version = 1
WHERE id = 1 AND version = 0

--Exception thrown
javax.persistence.RollbackException: Error while committing the transaction
Caused by: javax.persistence.OptimisticLockException:
Caused by: org.hibernate.StaleStateException:
Batch update returned unexpected row count from update [0];
actual row count: 0; expected: 1
```

Because this example uses the Java Persistence API, the Hibernate internal `StaleStateException` is wrapped in the `OptimisticLockException` defined by the JPA specification.

The flow of operations can be summarized as follows:

- Alice fetches a `Post` entity and then her thread is suspended.
- Bob thread is resumed, he fetches the same `Post` entity and changes the `title` to `High-Performance JDBC`. The entity `version` is set to 1.
- When Alice's thread is resumed, she tries to update the `Post` entity `title` to `High-Performance Hibernate`.
- An `OptimisticLockException` is thrown because the second update statement is expecting to filter the entity version with a value of 0, while the `version` column value is now 1.

16.1.1.1 Resolving optimistic locking conflicts

While pessimistic locking prevents conflict occurrences, optimistic locking mechanisms, just like MVCC, use conflict detection instead. So anomalies are detected and prevented from being materialized by aborting the currently running transactions, and Hibernate optimistic locking can prevent the *lost update* anomaly.

As explained in the application-level transactions section, when using a multi-request workflow, the database isolation level can no longer prevent lost updates. On the other hand, the optimistic locking mechanism can prevent losing updates as long as the entity state is preserved from one request to the other.

Optimistic locking discards all incoming changes that are relative to a stale entity version. However, everything has its price and optimistic locking is no different.

If two concurrent transactions are updating distinct entity attribute subsets, then there should be no risk of losing any update. However, the optimistic concurrency control mechanism takes an all-or-nothing approach even for non-overlapping changes. For this reason, two concurrent updates, both starting from the same entity version, are always going to collide. It is only the first update that is going to succeed, the second one failing with an optimistic locking exception.

This strict policy acts as if all changes are going to overlap, and, for highly concurrent write scenarios, the single version strategy can lead to a large number of transactions being rolled back.

To visualize the non-overlapping conflict, consider the following `Post` entity class:

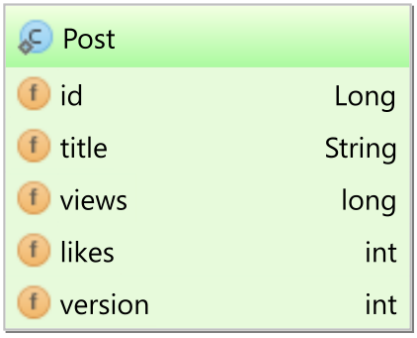

Figure 16.1: Post entity with a single global version

In the following example, Alice modifies the `Post` entity `title` attribute, Bob increments the `likes` counter, and Carol sets the `views` attribute to a value that was aggregated from an external batch processor.

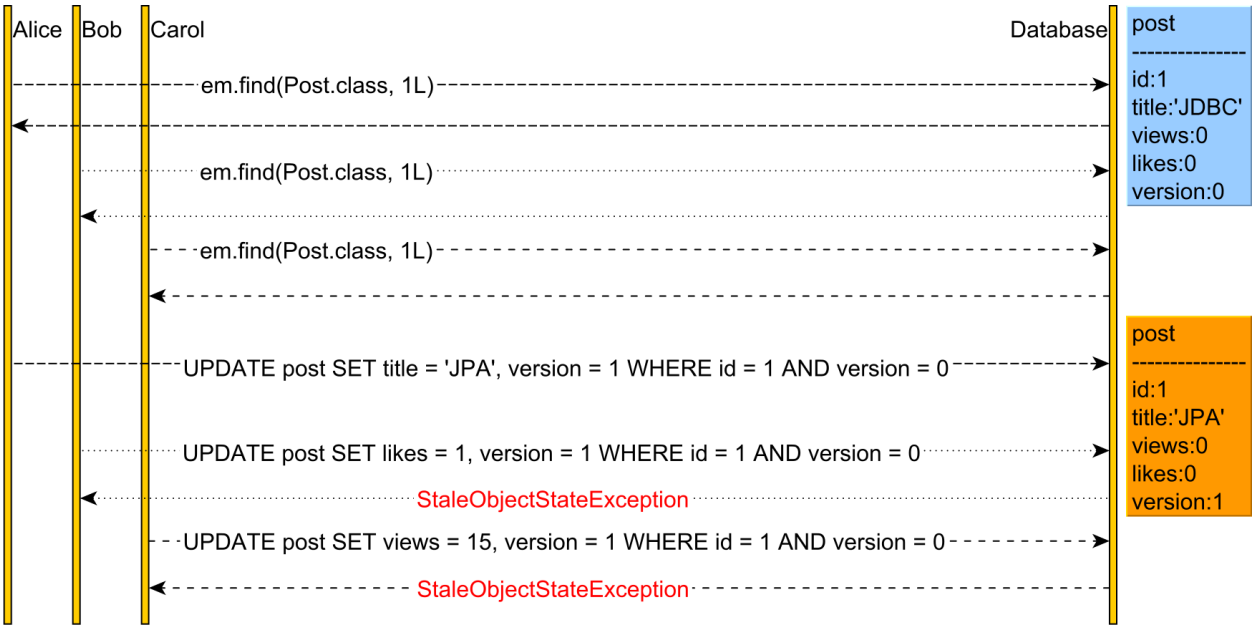

Figure 16.2: Optimistic locking non-overlapping conflict

The flow of operations can be explained as follows:

- All three users are loading the same `Post` entity version.
- Alice modifies the `title`. Therefore, the `Post` entity `version` is incremented.
- Bob tries to increment the `likes` counter but rolls back because it expects the `version` to be 0, but now it has a value of 1.
- Carol's transaction is also aborted because of the entity version mismatch.

The optimistic locking mechanism allows only monotonically increasing version updates. If changes were dependent one to another, then getting an `OptimisticLockException` is less of an issue than losing an update. However, if from a business logic perspective, the changing attributes are not overlapping, having a single global version is no longer sufficient.

For this reason, the single global version must be split into multiple subversions, and this can be done in two ways:

- Instead of having a single optimistic locking counter, there can be a distinct version for each individual attribute set.
- Each changing attribute can be compared against its previously known value, so lost updates are relative only to the attribute in question.